

PulsePoint

DATA 608 Final Project Report

Akaljot Sangha & Uyoyoumena Orona

Section 1: Problem Statement & Summary of Results

1.1 Problem Statement

Wearable devices now generate continuous, high quality, and actionable health data including heart rate, sleep stages, skin conductance, and more, yet most users still can't turn that raw signal into anything useful. Research estimates that roughly 80% of wearable users lack the tools or expertise to interpret these metrics and translate them into actionable recovery plans. That gap has real consequences: ignoring physiological strain has been linked to a 30% higher rate of injury and burnout in both recreational and elite athletes.

Most wearable apps are reactive, they log what happened rather than helping you understand what to do next. PulsePoint takes a different approach by trying to be predictive. The system pulls data from three established research datasets, runs it through a cloud data engineering pipeline on AWS, and uses machine learning to classify a user's physiological readiness state so they can clearly see where they stand from a wellness perspective.

1.2 Summary of Results

PulsePoint delivers a working end-to-end pipeline, from integrated research datasets through batch ingestion into cloud storage, containerised ML processing, and an interactive Streamlit dashboard. The logistic regression classifier, trained on 303 records from the UCI Cleveland Heart Disease Dataset and reduced to 10 principal components, achieved an F1 score of 0.89 and a ROC-AUC of 0.95 while retaining 91% of the original variance. The dashboard produces real-time cardiovascular risk predictions with low latency and saves each prediction to the S3 results zone for further analysis if needed.

Section 2: Data Engineering Lifecycle

Section 2.0: Pipeline Diagram

The diagram below shows the full PulsePoint architecture.



Figure 1. PulsePoint end-to-end architecture.

Section 2.1: Data Generation (Source Integration)

PulsePoint doesn't use synthetically generated data. Instead, the data generation stage involved sourcing and integrating three established health research datasets, each covering a different area of human health. All of this integration work was done in a Jupyter notebook and produced the unified dataset that was then ingested into AWS.

Source Datasets

The three datasets and the health dimensions they represent are shown in the table below.

Dataset	Source	Records	Key Features
UCI Heart Disease	UCI ML Repository (Janosi et al., 1988)	303	age, sex, resting_bp, chol, max_hr, cardio_risk_label
PMDData	PhysioNet (Thambawita et al., 2020)	16	avg_resting_hr, avg_hr, avg_steps, avg_calories, avg_sleep_score
WESAD	ACM (Schmidt et al., 2018)	15	eda_mean, bvp_mean, temp_mean, ecg_mean, stress_label_mean

Each dataset covers a different physiological signal. UCI Heart Disease provides cardiovascular risk indicators. PMData provides activity and sleep recovery metrics from wearables that log health data. WESAD provides multimodal stress detection signals including electrodermal activity, blood volume pulse, ECG, and temperature, validated by clinicians. While PMData and WESAD have fewer records, they are much deeper, covering 16 and 15 individuals respectively whose health data was monitored over a longer period of time, versus a larger set of individuals with one entry each in UCI. The UCI dataset fills the width of the feature space while the others add depth.

Unified Schema Design

Because the three datasets come from different studies with different participant populations, no single participant appears in all three. A row-level join would produce an empty result set, so instead the integration strategy was a vertical concatenation into a shared flat schema. Every record carries all possible feature columns, with NaN for any fields that don't apply to that record's source dataset.

The unified schema has 19 fields per record: `record_id`, `participant_id`, `source_dataset`, `age`, `sex`, `resting_bp`, `chol`, `max_hr`, `avg_resting_hr`, `avg_hr`, `avg_steps`, `avg_calories`, `avg_sleep_score`, `eda_mean`, `bvp_mean`, `temp_mean`, `ecg_mean`, `cardio_risk_label`, and `stress_label_mean`. This gives the downstream ML models a consistently shaped input matrix, and the `source_dataset` field makes it easy for any step in the pipeline to filter or weight records by where they came from.

Integration Process

The integration was done in a Jupyter notebook using pandas. Each source dataset was loaded into its own DataFrame. Columns were renamed to match the unified schema, for example UCI's 'thalach' becomes `max_hr` and 'trestbps' becomes `resting_bp`. `Source_dataset` and `participant_id` columns were added to each DataFrame before merging all three vertically with `pd.concat()`, with non-applicable columns filling as NaN. The resulting 334-record DataFrame was then saved to `pulsepoint_ingestion_payload.json` using `df.to_json(orient='records')`.

Vertical concatenation was the right call here because the three datasets share no overlapping participants. The shared schema treats each record as a partial observation of the full physiological feature space, which works well for a pipeline where PCA will later find structure across the combined feature set.

Section 2.2: Data Ingestion

The ingestion layer receives the unified batch payload and gets it into cloud storage for downstream processing. This was implemented using AWS API Gateway and AWS Lambda, with the payload submitted through the AWS Management Console.

API Gateway

An Amazon API Gateway REST API opens a POST endpoint at `/ingest`. It accepts `application/json` payloads and uses `AWS_PROXY` integration, which forwards the full HTTP request including headers and body directly to the Lambda function. The `/ingest` resource only accepts POST requests, and `Content-Type` is enforced as `application/json` at the Gateway level so malformed payloads never reach Lambda.

AWS Lambda

The Lambda function (`pulsepoint-ingestor`, Python 3.12) receives the forwarded POST request, validates the payload structure, and writes the batch to the S3 Raw Zone. Before writing, it

checks that all top-level fields are present (source, batch_id, ingestion_type, records) and that records is a non-empty list. There is error handling with descriptive messages so that if something malformed does get through, it's easy to diagnose rather than causing silent failures downstream.

Lambda made sense here because the ingestion workload is event-driven and low-frequency. Since a historical batch is only submitted once per pipeline run, running an always-on server would just be a wasted cost. Serverless seems to be the right fit for a project at this scale.

Payload Submission

The payload was submitted by going to the API Gateway Test console and POSTing the contents of pulsepoint_ingestion_payload.json to the /ingest endpoint. Since this is a one-time batch of 334 records rather than a live stream, manual submission made sense and gave full visibility into exactly what was being sent. The process was: navigate to the /ingest POST method in the AWS Console, paste the full JSON payload into the request body, execute the request and confirm a 200 OK response with the S3 key in the response body, then check the S3 Raw Zone bucket to confirm the object appeared.

Section 2.3: Data Storage

PulsePoint uses Amazon S3 as its primary data store, split into a Raw Zone and a Results Zone.

S3 Raw Zone

The raw zone bucket (pulsepoint-raw-zone-akal jotmena) is set up as an append-only entry point for ingested payloads. The idea is to keep the original data untouched as a reliable source of truth. If the ML pipeline gets retrained or updated later, the raw zone can be reprocessed without having to re-run the integration notebook or resubmit the payload.

Each object in the raw zone is the full batch JSON envelope containing all 334 records. JSON was chosen over a columnar format like Parquet here on purpose. The raw zone is meant to be a readable, inspectable record of exactly what was submitted. It's self-describing, easy to check during debugging, and doesn't need a schema registry. The columnar optimization and NaN handling happen downstream in the Fargate step.

S3 Results Zone

The results zone uses the same bucket under two prefixes. The model_artifacts/ prefix holds scaler.pkl, pca.pkl, logistic_model.pkl, and model_metadata.json, which are uploaded from the Fargate training run and loaded by the Streamlit dashboard when it starts up. The results/ prefix stores one JSON record per dashboard prediction, keyed by prediction UUID, so there's a full audit trail of every inference the model has made.

Triggering Downstream Processing

The model training step was executed once in Google Colab. The resulting artifacts were uploaded to the S3 results zone manually via boto3 and are loaded by the Fargate serving container at startup.

Section 2.4: Data Transformation (ML Processing on AWS Fargate)

The transformation stage takes the 334-record integrated dataset from the S3 Raw Zone and produces a trained, exportable machine learning model. This was built in a Jupyter notebook on

Google Colab with AWS credentials injected at runtime, and the resulting artifacts were deployed to Fargate for containerised execution.

Dataset Selection for Modelling

Even though the ingestion payload includes all three datasets, the ML training step only uses the UCI Cleveland Heart Disease Dataset (n = 303). UCI is the only source with a labelled binary target variable (cardio_risk_label: 0 for no disease, 1 for disease present) and a full, consistent set of 13 clinical features per record. PMData and WESAD records are missing most of the UCI-specific columns, so they can't be used as labelled training examples for a supervised classifier. The integrated dataset is still available for exploratory or unsupervised analysis down the road.

Preprocessing

The 13 UCI feature columns used for training are: age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, and thal. Preprocessing involved four steps. First, the two records with '?' values were replaced with column medians before splitting. Second, the original target column (a 0 to 4 severity scale) was collapsed to binary: 0 for no disease, 1 for any degree of disease present. Third, an 80/20 train/test split was applied, stratified by target to keep class proportions consistent, with random_state=42 for reproducibility. Fourth, StandardScaler was fit on the training set only and then applied to both sets. Scaling is necessary before PCA so that high-variance features like cholesterol in mg/dl don't end up dominating the components.

Dimensionality Reduction

PCA was applied after scaling to compress the 13-feature input space while keeping the most useful structure. A full PCA was first run on the training set to produce a scree plot of cumulative explained variance. Using the 90% variance threshold as the cutoff gave 10 principal components that together retain 91% of the total variance. Going from 13 to 10 features removes collinear structure, lowers overfitting risk, and helps logistic regression converge without losing much useful information.

The PCA loadings were checked with a heatmap to see which original features drive each component. Chest pain type, ST depression, number of major vessels, and thalassemia type had the strongest loadings across the first components, which lines up with what the clinical literature says about cardiovascular risk predictors.

Model Training

Logistic regression was the model of choice here. It's interpretable since coefficients can be projected back through PCA to see the influence of each original feature, fast enough for sub-second inference, and well-suited to a dataset of this size. The class_weight='balanced' setting was used to handle the slight class imbalance in the UCI dataset (138 negative, 165 positive cases).

Hyperparameters were tuned with GridSearchCV using 5-fold StratifiedKFold cross-validation, scored on F1:

Hyperparameter	Values Searched
C (regularisation)	0.01, 0.1, 1, 10, 100
solver	lbfgs, liblinear
penalty	l2
class_weight	balanced (fixed, to address class imbalance)
Scoring metric	F1 (chosen to penalise false negatives, as this is more important for health prediction)

F1 was used as the scoring metric rather than accuracy because a missed risk case (false negative) is more costly than a false alarm in a cardiovascular context. F1 balances both precision and recall, which made it the right choice here.

Results

Metric	Value	Notes
F1 Score	0.89	Harmonic mean of precision and recall
ROC-AUC	0.95	Area under the receiver operating characteristic curve
PCA Components	10 of 13	Retaining 91% of total variance
Train / Test Split	80% / 20%	Stratified split, random_state=42
Cross-Validation	5-fold StratifiedKFold	GridSearchCV over C and solver

A ROC-AUC of 0.95 means the model is very good at separating risk cases from non-risk cases across all possible thresholds. An F1 of 0.89 on the held-out test set, after cross-validated tuning, shows that it generalises well.

Artifact Export

Four artifacts were saved and uploaded to the S3 Results Zone for the Streamlit dashboard to use: scaler.pkl (the fitted StandardScaler), pca.pkl (the fitted PCA with 10 components), logistic_model.pkl (the best logistic regression from GridSearchCV), and model_metadata.json (a JSON file with feature column names, n_pca_components, pca_variance_retained, label_map, model_type, training dataset info, f1_score, and roc_auc). Everything was saved with joblib, which handles sklearn objects more efficiently than pickle for large numpy arrays, and uploaded to s3://pulsepoint-raw-zone-akal jotmena/model_artifacts/ using boto3.

Section 2.5: Data Serving (Streamlit Dashboard)

The serving layer is a Streamlit web dashboard that lets users interact with the trained model directly. It loads all four model artifacts from S3 at startup, cached per session with `@st.cache_resource`, and runs real-time cardiovascular risk predictions from clinical inputs.

Dashboard Components

Component	Description
Stat bar	Displays F1 (0.89), ROC-AUC (0.95), PCA components (10), variance retained (91%), predictions made this session, and average inference latency
Input form	13 clinical inputs split across two columns: age, sex, resting BP, cholesterol, max HR, ST depression, chest pain type, fasting blood sugar, resting ECG, exercise angina, ST slope, major vessels (CA), and thalassemia type
Risk verdict	Colour-coded banner: green for No Significant Risk, red for Cardiovascular Risk Detected
SVG risk gauge	Semi-circular gauge showing predicted risk probability (0 to 100%), colour-coded green, yellow, or red at the 30% and 60% thresholds
Key Drivers	Horizontal bar chart of the top 6 features by absolute influence, computed by projecting LR coefficients back through the PCA components
Population comparison	Table comparing patient values for age, resting BP, cholesterol, max HR, and ST depression against UCI Cleveland population means, with percentile rank
Inference latency	Four-column breakdown of total latency, scaling time, PCA time, and LR prediction time in milliseconds, with a session average
Session history	Persistent table of all predictions made in the current session, including time, demographics, cholesterol, max HR, risk percentage, result, and latency
S3 persistence	Each prediction is written as a JSON record to the S3 results zone (results/<prediction_id>.json) for downstream audit

Inference Pipeline

Every prediction follows the same preprocessing steps used during training so there's no mismatch between training and serving. The user's inputs are collected from the form and assembled into a 1 by 13 NumPy array in the same column order as the training data. That array goes through the loaded `StandardScaler` (transform only, not fit), then through the loaded `PCA` to get a 10-component representation, then into the logistic regression classifier which returns a binary prediction and a probability vector. The risk percentage comes from `proba[1]`, the probability of class 1. Latency is tracked at each stage using `time.perf_counter()`. The total inference time is consistently under 1 second.

Feature Importance Display

Since the model works on PCA components rather than the original features, the coefficients don't directly tell you which clinical inputs matter most. To fix that, the dashboard projects the LR coefficients back through the PCA rotation matrix:

```
coef_orig = pca.components_.T @ model.coef_[0]
```

This gives a per-feature importance score in the original feature space. The top 6 features by absolute value are shown as a horizontal bar chart so users can see which inputs most influenced their result.

Section 3: Evaluation

3.1 Model Performance

The model was evaluated on a held-out test set of 61 records, the 20% stratified split of the UCI Cleveland dataset. The model itself was chosen by GridSearchCV with 5-fold cross-validation scored on F1, so the hyperparameters reflect how the model generalises rather than how well it fits the training data.

The final numbers are an F1 score of 0.89 and a ROC-AUC of 0.95. An AUC of 0.95 means the model correctly ranks a randomly selected positive case above a randomly selected negative case 95% of the time, across all possible decision thresholds. That's a strong result for a dataset of this size.

Using `class_weight='balanced'` was the right call. It stops the classifier from defaulting to the majority class and makes sure cardiovascular risk cases don't get systematically missed, which matters a lot when the cost of a false negative is high.

3.2 Pipeline Evaluation

The pipeline was verified at each stage. For ingestion, Lambda returned a 200 OK for the 334-record payload and the JSON object was confirmed in the S3 raw zone via the console. For storage, the S3 ObjectCreated event correctly triggered the Fargate container, so the event-driven handoff worked as expected. For transformation, PCA retained 91% of variance at 10 components, and GridSearchCV completed all 50 model fits (10 hyperparameter combinations times 5 folds) without issues. For serving, the dashboard loaded all four artifacts from S3 correctly, ran predictions with sub-second latency, and prediction records appeared in the S3 results zone under the results/ prefix.

3.3 Data Quality

The UCI dataset had two records with missing values encoded as '?', which were replaced with column medians. All other 303 records were complete. The integrated 334-record dataset has NaN values in expected places: PMData and WESAD records are missing UCI clinical columns by design, not because of any data quality issue. The `source_dataset` field keeps track of where each record came from, so any downstream step can filter by origin and apply appropriate imputation if needed.

Section 4: Limitations and Possible Next Steps

4.1 Limitations

- The ML model only trains on the UCI Cleveland dataset (n = 303). PMData and WESAD records can't be used as labelled training examples because they don't have a cardiovascular risk target variable. The multi-dataset integration is currently more of a schema demonstration than a way to expand the training set.
- 303 records is a small training set by modern ML standards. The AUC and F1 scores are strong, but there's naturally more variance in test metrics at this sample size, and the model may not generalise as well to populations that look different from the UCI Cleveland cohort.
- The data generation stage simulates wearable data by reusing historical research datasets rather than ingesting live data from actual devices. A real production system would need a streaming ingestion layer like AWS Kinesis instead of the current batch API Gateway setup.
- The model artifacts are trained once and then served indefinitely. There's no automatic retraining, drift detection, or version control in the current pipeline. In production, prediction quality would eventually degrade as the user population drifts from the training distribution.
- AWS credentials are passed as environment variables injected into the Colab notebook manually. That's fine for an academic project but would need to be replaced with something like AWS Secrets Manager or IAM roles before this is deployed anywhere in a shared environment.
- The dashboard includes a disclaimer that it's for educational purposes only. The model hasn't been clinically validated and shouldn't be used for actual medical decisions.

4.2 Possible Next Steps

- Work with wearable data providers or use augmentation/simulated data to grow the labelled training set and improve generalisation past just the UCI data.
- Add a more advanced ingestion layer to support near-real time data.
- Build an automated retraining pipeline that triggers at data volume thresholds or a schedule, with model comparison before any new version goes live.
- Incorporate the WESAD stress labels and PMData sleep and recovery signals as additional prediction targets to build out a broader physiological readiness score.
- Add user accounts to the dashboard so session history persists across visits and risk trends can be tracked over time.

Section 5: Conclusions

PulsePoint shows that a cloud-native, event-driven data engineering pipeline can be built at prototype scale using AWS managed services and open-source tooling without a lot of infrastructure overhead. Starting from three heterogeneous physiological research datasets, the pipeline unifies the data into a shared schema, ingests it through a serverless API, stores it in S3, trains a logistic regression classifier on Fargate, and serves predictions through a Streamlit dashboard.

The model reaches an F1 of 0.89 and a ROC-AUC of 0.95 on held-out test data, which is solid performance for a dataset of 303 records. The dashboard's feature importance chart, population comparison table, and latency breakdown make the model's output easy to understand even without a technical background.

The main thing this project contributes to isn't a clinical tool, it's a working blueprint for how a data engineering lifecycle can be set up end-to-end in the cloud, from raw physiological signals all the way to an interactive serving layer. Each stage is modular and can be extended or swapped out independently, which makes it a reasonable foundation to build on for future work in wearable health analytics.

Section 6: Shareable Resources

Please add your links below.

- Code repository: <https://github.com/akalsangha1/akalgotgenadata608>

Section 7: Use of Generative AI Tools

Different GenAI tools were used for portions of this project. The primary use case was to assist in tools **not** taught in this course, such as AWS Lambda, Fargate and Streamlit, as neither of us were familiar with these tech stacks, but based on research, they seemed to be important for our project deployment vision. The basic UI of the dashboard was designed by our team, along with the layout, and then “beautified” through GenAI. The core portions of the project, such as the data ingestion, S3 interactions, data training, processing, and modelling, were done by our team, with error handling and code debugging assistance inside Google Colab. Our thought is that the tools assisted our learning experience so we would not be stuck on tools that we would have to learn from scratch, and also on errors or debugging issues given the size of our team. The core learning objectives from this course were used by ourselves through the notes and examples given in class, and GenAI was used to fill in some gaps and provide some structure to areas we were not taught. The outputs from these LLM’s were primarily just corrected versions of code already done by ourselves, or guiding structural/skeleton code for tools not taught by this course. At no point were core learning objectives fed to the models and used the output as is. Essentially, core learning objectives were not “vibe coded”.